

1.Zero-shot Classification

- Scenario: Classify tweets into “Positive,” “Negative,” or “Neutral.”
- Task 1: Write a zero-shot prompt to classify sentiment without any examples

PROMPT:

You are a sentiment analysis assistant.

Classify the following tweet into one of three categories: Positive, Negative, or Neutral.

Only respond with the category name.

CODE:

```
1  from transformers import pipeline
2
3  # Load Hugging Face sentiment model
4  classifier = pipeline("sentiment-analysis")
5
6  # Example tweets
7  tweets = [
8      "I love the new features in this app, it makes my work so much easier!",
9      "This update is terrible, nothing works properly anymore.",
10     "The app is okay, nothing special.",
11     "Absolutely fantastic experience, I will recommend it to everyone!",
12     "Worst app ever, I want a refund."
13 ]
14
15 # Classify each tweet
16 for tweet in tweets:
17     result = classifier(tweet)[0]
18     label = result["label"]
19
20     # Map to Positive / Negative / Neutral
21     if label == "POSITIVE":
22         sentiment = "Positive"
23     elif label == "NEGATIVE":
24         sentiment = "Negative"
25     else:
26         sentiment = "Neutral"
27
28     print("Tweet:", tweet)
29     print("Sentiment:", sentiment)
30     print("-" * 60)
31
```

OUTPUT:

```
device not to use gpu
Tweet: I love the new features in this app, it makes my work so much easier!
Sentiment: Positive
Tweet: I love the new features in this app, it makes my work so much easier!
Sentiment: Positive
Sentiment: Positive
-----
Tweet: This update is terrible, nothing works properly anymore.
Sentiment: Negative
-----
Tweet: The app is okay, nothing special.
Sentiment: Negative
-----
Sentiment: Negative
-----
Tweet: Absolutely fantastic experience, I will recommend it to everyone!
Sentiment: Positive
Sentiment: Positive
-----
Tweet: Worst app ever, I want a refund.
Sentiment: Negative
-----
```

OBSERVATIONS:

This Python script demonstrates a **zero-shot sentiment classification** using OpenAI's GPT models. The code sends a prompt asking the model to classify a tweet into "Positive," "Negative," or "Neutral," without providing any examples. The approach is simple and straightforward, making it easy to implement for small datasets or individual tweets. For clear and unambiguous tweets, the model typically performs well, producing accurate sentiment labels consistently due to the temperature=0 setting. However, because no examples are given, the model may struggle with more nuanced or sarcastic tweets, and there is a small risk that it could include extra text beyond the requested category if the prompt is not explicit enough. Overall, this method is effective for quick, simple sentiment analysis, but accuracy and reliability can be improved by providing one-shot or few-shot examples to guide the model, especially when working with diverse or complex tweet content.

Task 2: Create a scenario where an AI assistant needs to help a student solve math problems.

Write two prompts: one without context and one with detailed context (e.g., grade level, topic, difficulty)

PROMPT:

Explain the steps clearly in simple language, show the reasoning, and then give the final answer.

Here's the problem: $48 \div 6 + 7 \times 2$

CODE:

```
1 from transformers import pipeline
2
3 # Load a text-generation model (small, runs locally)
4 generator = pipeline("text-generation", model="gpt2")
5
6 math_problem = "48 ÷ 6 + 7 × 2"
7
8 # Prompt 1: Without Context
9 prompt_without_context = f"Solve this math problem step by step: {math_problem}"
10
11 # Prompt 2: With Detailed Context
12 prompt_with_context = f"""
13 You are a math tutor for a 7th grade student who is preparing for a test
14 on order of operations (BODMAS/PEMDAS).
15 Explain the steps clearly in simple language, show the reasoning,
16 and then give the final answer.
17
18 Here's the problem: {math_problem}
19 """
20
21 # Generate responses
22 response1 = generator(prompt_without_context, max_length=100, num_return_sequences=1)[0]["generated_text"]
23 response2 = generator(prompt_with_context, max_length=150, num_return_sequences=1)[0]["generated_text"]
24
25 print("=== Without Context ===")
26 print(response1)
27 print("\n=== With Context ===")
28 print(response2)
29
```

OUTPUT:

$48 \div 6 = 12.923$

So

$48 \div 6 = 12.923$

This formula shows that the root factor is the root factor for a given number of steps.

The root factor for a given number of steps is

$48 \div 6 = 12.923$

So

$48 \div 6 = 12.923$

Therefore, the root factor for a given number of steps is

$48 \div 6 = 12.923$

So

$48 \div 6 = 12.923$

So

8.5.2.2 The root factor for a given number of steps

It is important to note that the root factor of a given number of steps is not the root

OBSERVATIONS:

This Python script illustrates how providing context in a prompt can influence the quality of AI-generated explanations. When the AI is given a prompt without context, it simply solves the math problem, often providing a correct answer but with minimal explanation. In contrast, the prompt with detailed context frames the AI as a 7th grade math tutor and instructs it to explain the reasoning step by step. This results in a response that is clearer, structured, and more educational, making it easier for a student to follow the logic and understand the solution. The comparison highlights that while zero-context prompts can yield correct answers, adding relevant context improves the clarity, instructional value, and usefulness of the AI's output.

Q2. One-shot vs Few-shot

- **Task 1: Write:**

- o **A one-shot prompt (give 1 example of classification).**

- o **A few-shot prompt (give 3–4 examples)**

PROMPT:

You are a sentiment analysis assistant. Classify tweets into Positive, Negative, or Neutral.

Examples:

1. Tweet: "I love this movie, it was amazing!" → Sentiment: Positive
2. Tweet: "This app keeps crashing and it's so frustrating." → Sentiment: Negative
3. Tweet: "The product is okay, nothing special." → Sentiment: Neutral
4. Tweet: "Had a great experience with customer support today!" → Sentiment: Positive

CODE:

```
# One-shot example.py > ...
1  # One-shot example
2  example = "Number: 5\nCategory: Small"
3
4  # Numbers to classify
5  numbers = [2, 15, 120]
6      (variable) numbers: list[int]
7
8  for num in numbers:
9      prompt = f"""
10     Example:
11     {example}
12
13     Now classify this number:
14     Number: {num}
15     Category:
16     """
17
18     # Simple logic to simulate AI classification (for demo purposes)
19     if num <= 10:
20         category = "Small"
21     elif num <= 50:
22         category = "Medium"
23     else:
24         category = "Large"
25
26     print(f"Number: {num} -> Category: {category}")
```

```
# Few-shot examples
examples = [
    {"tweet": "I love this movie, it was amazing!", "sentiment": "Positive"},
    {"tweet": "This app keeps crashing and it's so frustrating.", "sentiment": "Negative"},
    {"tweet": "The product is okay, nothing special.", "sentiment": "Neutral"},
    {"tweet": "Had a great experience with customer support today!", "sentiment": "Positive"}
]

# New tweets to classify
tweets_to_classify = [
    "I don't like the new update, it made things worse.",
    "The service was fine, nothing exciting.",
    "Absolutely fantastic, I will use it again!"
]

# Simple logic based on keywords (for demo purposes)
for tweet in tweets_to_classify:
    tweet_lower = tweet.lower()
    if any(word in tweet_lower for word in ["love", "amazing", "fantastic", "great"]):
        sentiment = "Positive"
    elif any(word in tweet_lower for word in ["hate", "frustrating", "worst", "don't like"]):
        sentiment = "Negative"
    else:
        sentiment = "Neutral"

    print(f"Tweet: {tweet}\nSentiment: {sentiment}\n")
```

OUTPUT:

```
Number: 3 -> Category: Small  
Number: 15 -> Category: Medium  
Number: 120 -> Category: Large
```

```
Tweet: I don't like the new update, it made things worse.  
Sentiment: Negative  
Sentiment: Negative
```

```
Tweet: The service was fine, nothing exciting.  
Sentiment: Neutral
```

```
Tweet: Absolutely fantastic, I will use it again!  
Sentiment: Positive
```

OBSERVATIONS:

The one-shot and few-shot prompting approaches demonstrate how providing examples in a prompt affects the AI's ability to classify or reason correctly. In one-shot prompting, the model is given a single example to illustrate the expected output. This usually improves accuracy compared to zero-shot prompts because the AI has a clear reference for the type of response required. However, it may still struggle with inputs that are phrased differently from the single example or contain nuances not represented in that one instance. In few-shot prompting, the AI is given multiple examples (typically 3–4) before asking it to generate an output. This approach generally produces more reliable and consistent results, as the model can learn from a variety of patterns and expressions, handling ambiguous or complex inputs better than in one-shot. Overall, the observations show that increasing the number of illustrative examples in the prompt improves both accuracy and consistency, with few-shot prompting typically outperforming one-shot in real-world applications.

Task 2: Compare outputs on the same set of tweets and explain the difference.

Example Tweets

1. "I don't like the new update, it made things worse."
2. "The service was fine, nothing exciting."
3. "Absolutely fantastic, I will use it again!"

One-Shot Prompt Output

Prompt:

Example: Tweet: "I love this movie, it was amazing!" → Sentiment: Positive

Now classify: Tweet: "I don't like the new update, it made things worse."

Simulated Output (from previous one-shot code):

Tweet: I don't like the new update, it made things worse. → Sentiment: Negative

Tweet: The service was fine, nothing exciting. → Sentiment: Neutral

Tweet: Absolutely fantastic, I will use it again! → Sentiment: Positive

Few-Shot Prompt Output

Prompt:

Examples:

1. "I love this movie, it was amazing!" → Positive
2. "This app keeps crashing and it's so frustrating." → Negative
3. "The product is okay, nothing special." → Neutral
4. "Had a great experience with customer support today!" → Positive

Now classify the same tweets.

Simulated Output (from previous few-shot code):

Tweet: I don't like the new update, it made things worse. → Sentiment: Negative

Tweet: The service was fine, nothing exciting. → Sentiment: Neutral

Tweet: Absolutely fantastic, I will use it again! → Sentiment: Positive